Project Report Template

Advanced Analytics and Applications

Team 1

Authors:
Ivan Kamal Mehieddin (7396805)
Nils Bringewatt (7443820)
Maike Katterbach (7444323)
Vitus Grofl (7380537)
Contact: mkatter1@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter
Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-04-01

Table of contents

List of Figures

1 Introduction

tbd

2 Data

We combine two data sources covering 2024-01-01 through 2026-05-31: Chicago Taxi Trips, the full set of reported trips pulled from the City of Chicago Data Portal (dataset `ajtu-isnz`), and hourly weather (temperature, precipitation, snowfall, wind, cloud cover) for Chicago from the Open-Meteo archive API. The taxi data cuts off at the end of May 2026 rather than the pull date because the portal has a reporting lag of several weeks — June 2026 held only 31 rows instead of the roughly 600,000 expected, which would otherwise look like a demand crash in the prediction models.

The raw taxi export contains 16,086,180 rows across 21 columns. The cleaning pipeline concatenates the 2024, 2025, and 2026 CSV exports, applies the steps below, and writes the result out as a single Parquet file. In total, 10.1% of trips (1,620,530 rows) are removed:

1. Trips are deduplicated on `trip_id` before any other filtering (775 rows removed), keeping the first occurrence. Rows without a `trip_id` are set aside first and rejoined afterward, since `drop_duplicates` would otherwise treat several genuine but ID-less trips as duplicates of each other.
2. Chicago reports location at two resolutions: `census_tract` (fine-grained) and `community_area` (77 coarse zones covering the whole city). Tracts are missing for over half of trips — not randomly, but because the city masks any tract with too few trips, for privacy. Community areas are far more complete (98.2% pickup, 91.6% dropoff), since they're only missing when a trip starts or ends outside city limits. Rather than dropping those out-of-city trips, we keep every row and add `pickup_flag`/`dropoff_flag` columns marking whether an area is present, so later spatial analyses can filter on the flags without losing the trips for everything else.
3. The centroid latitude/longitude columns silently mix resolutions: where a tract exists, they hold its centroid; where it doesn't, they fall back to the community area's centroid. We derive a clean area-to-centroid mapping (from exactly the rows where the raw coordinate already is the area centroid) and join it back as separate `*_ca_centroid_lat/lon` columns, giving a resolution-consistent coordinate for any area-level aggregation.
4. Trips missing `fare`, `trip_total`, `trip_seconds`, or `trip_miles` are dropped (34,687 rows), since no meaningful analysis is possible without them.
5. Negative `tips/tolls/extras` would be removed, though none occur in practice — zero is legitimate (e.g. cash trips report no tip). Trips below Chicago's legal minimum fare ($3.25 flag-drop) or shorter than 60 seconds are treated as test or aborted entries and removed, together with trips above the 99.9th percentile of distance (43.0 miles) or duration (165 minutes). These bounds account for the bulk of all removed rows (1,585,068 of 1,620,530).

`payment_type` and `company` are left untouched throughout; their missing values remain as `NA` and are handled per analysis.

The cleaned dataset has 14,465,650 rows and 27 columns, spanning 2024-01-01 to 2026-05-31.

3 Demand Prediction — Support Vector Machines

3.1 Modeling Approach

We predict taxi pickup demand — the number of trips originating in a spatial unit within a fixed time window — using Support Vector Regression (SVM/SVR). Rather than fitting one model per spatial unit, we train a single **pooled model per resolution**: all units are trained together, with spatial identity encoded as fixed-effect dummy variables rather than as separate models. This design choice is motivated by scalability. Fitting hundreds of independent models — one per census tract, for instance — would multiply both training time and maintenance effort, whereas a pooled model reuses the same panel-construction and evaluation code across resolutions and requires only a single fit per experiment.

The feature set consists of 17 exogenous time and weather variables, including two harmonics (sine/cosine pairs) each for hour-of-day and day-of-week, which allow the model to represent the bimodal morning and evening rush pattern that a single harmonic cannot capture. Community areas are represented through one-hot dummy variables. Per the constraints of this course, no lagged or autoregressive demand features are used; the model relies exclusively on exogenous inputs that would be known in advance of a prediction.

Because raw demand counts are heavily right-skewed — median demand per area spans roughly three orders of magnitude, from single digits to several hundred trips per window — the regression target is `log1p(count)`, with predictions back-transformed via a clipped `expm1`. Without this transformation, squared-error loss would be dominated by a small number of high-demand windows, and the model would effectively ignore the majority of lower-demand areas.

3.2 Validation Strategy

Model evaluation follows a chronological holdout: the training window spans January 2024 through December 2025, and the test window spans January through May 2026. All rows are drawn from a complete, zero-filled grid of every spatial unit crossed with every time window, so that models are evaluated on genuine zero-demand periods rather than only on windows with observed trips. Hyperparameter search uses `TimeSeriesSplit` with three folds, preserving temporal order within the training data.

Because absolute error metrics such as MAE or RMSE are not comparable across areas of very different demand magnitude, we introduce a **profile baseline** — the train-set mean demand per hour-of-day and weekday combination, computed separately for each spatial unit — as the reference model against which every SVM variant is judged. Performance is then reported as a **skill score**, defined as $1 - \mathrm{MAE}_{\mathrm{model}}/\mathrm{MAE}_{\mathrm{baseline}}$, computed per unit. Because it is a ratio, each unit's own demand scale cancels out, making the score comparable across units of very different volume. Skill scores are aggregated in two complementary ways: a demand-weighted mean, which serves as the headline metric because it matches a fleet-allocation use case where high-demand areas matter most, and an unweighted median, which serves as a guard rail against a model that only performs well on a handful of large units.

3.3 Results: Community Area, 4-Hour Resolution

The profile baseline achieves a pooled R^2 of 0.933 across all 77 community areas, but its per-area median R^2 is considerably lower, at 0.304, and 14 of the 77 areas have a negative per-area R^2. These are predominantly low-demand areas, where near-zero trip counts leave little systematic variance for any model — baseline or otherwise — to explain.

A pooled `LinearSVR` fit on the full 337,722-row training set achieves a pooled R^2 of 0.778, which appears competitive at first glance. However, its demand-weighted skill relative to the baseline is **-0.75**, meaning it systematically underperforms the baseline specifically in the highest-demand areas, even though it wins on a majority of individual areas by the unweighted median (skill of -0.03). This asymmetry arises because every area contributes an equal number of training rows regardless of its demand level, so the model's shared coefficients are fit predominantly to the low-demand majority. We attempted to correct this by refitting with sample weights proportional to the square root of each area's median demand, prioritizing high-demand areas during training the same way they are prioritized during evaluation. This improved the demand-weighted skill to -0.61, but at the cost of degrading the majority of areas — 67 of 77 areas now show negative skill, compared to 48 of 77 in the unweighted case — so weighting is better characterized as a trade-off than a resolution of the underlying issue.

We also evaluated kernelized SVR (RBF, polynomial, and sigmoid kernels). Because exact kernel SVR scales quadratically to cubically with the number of training rows, kernel models were tuned on a random 10,000-row subsample of the training data and evaluated against the full test set. These results proved highly unstable: hyperparameters tuned via `GridSearchCV` achieved a cross-validated R^2 of only 0.088 for the polynomial kernel, yet the same configuration scored $R^2 = -0.032$ on the actual held-out test set — worse than the baseline for 62 of 77 areas — despite an untuned polynomial kernel appearing to perform excellently ($R^2 = 0.875$) on a single earlier draw of the subsample. We conclude that tuning kernel SVR from a small subsample of a much larger dataset is too high-variance to be reliable at this resolution.

As a further structural attempt to address the high-demand shortfall, we fit fully separate per-area models — no pooling and no shared coefficients — for the 20 areas with median demand of at least 10 trips per window. Even without any cross-area dilution, most configurations still lose to the profile baseline: default per-area `LinearSVR` and RBF models score -0.10 and -0.11 on demand-weighted skill, respectively, and per-area hyperparameter tuning only partially closes the gap. The sole configuration across the entire study that achieves a positive demand-weighted skill is a fully tuned per-area RBF SVR, at +0.023 — a small margin, and one under which 12 of the 20 areas are still individually negative.

Taken together, these results point to a consistent pattern: across every variant we tried — pooled, demand-weighted, demand-tiered, and per-area — the profile baseline proves a surprisingly strong and difficult-to-beat reference specifically where demand is largest. We attribute this to the interaction between the `log1p` target transformation and its `expm1` back-transform, which amplifies error multiplicatively rather than additively: a small error in log-space becomes a large absolute error precisely for the highest-count predictions, a penalty the baseline, which predicts directly on the raw count scale, never incurs. Full methodological detail, diagnostic visualizations, and per-area breakdowns are available in `notebooks/03.05_prediction_svm_pooled copy.ipynb`.

3.4 Results: Census Tract Resolution

To be added. Panel construction and model code for the census-tract resolution are already implemented in `notebooks/03.06_prediction_svm_tract.ipynb`, reusing the panel-building and evaluation logic developed for the community-area resolution above. Results are pending a completed model run and will be reported in this section.

3.5 Results: 1-Hour Resolution

To be added.

4 Citations and References

You can easily cite your sources using BibLaTeX. For example, you can cite the Quarto documentation (Team, 2024) or a textbook like "R for Data Science" (Wickham et al., 2023). The bibliography will be automatically generated at the end of the document.

5 Conclusion

The conclusion should summarize your findings and suggest future work.

References

Team, Q. (2024). Quarto: An open-source scientific and technical publishing system. *Journal of Modern Data Science*. https://quarto.org

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for data science*. O'Reilly Media.